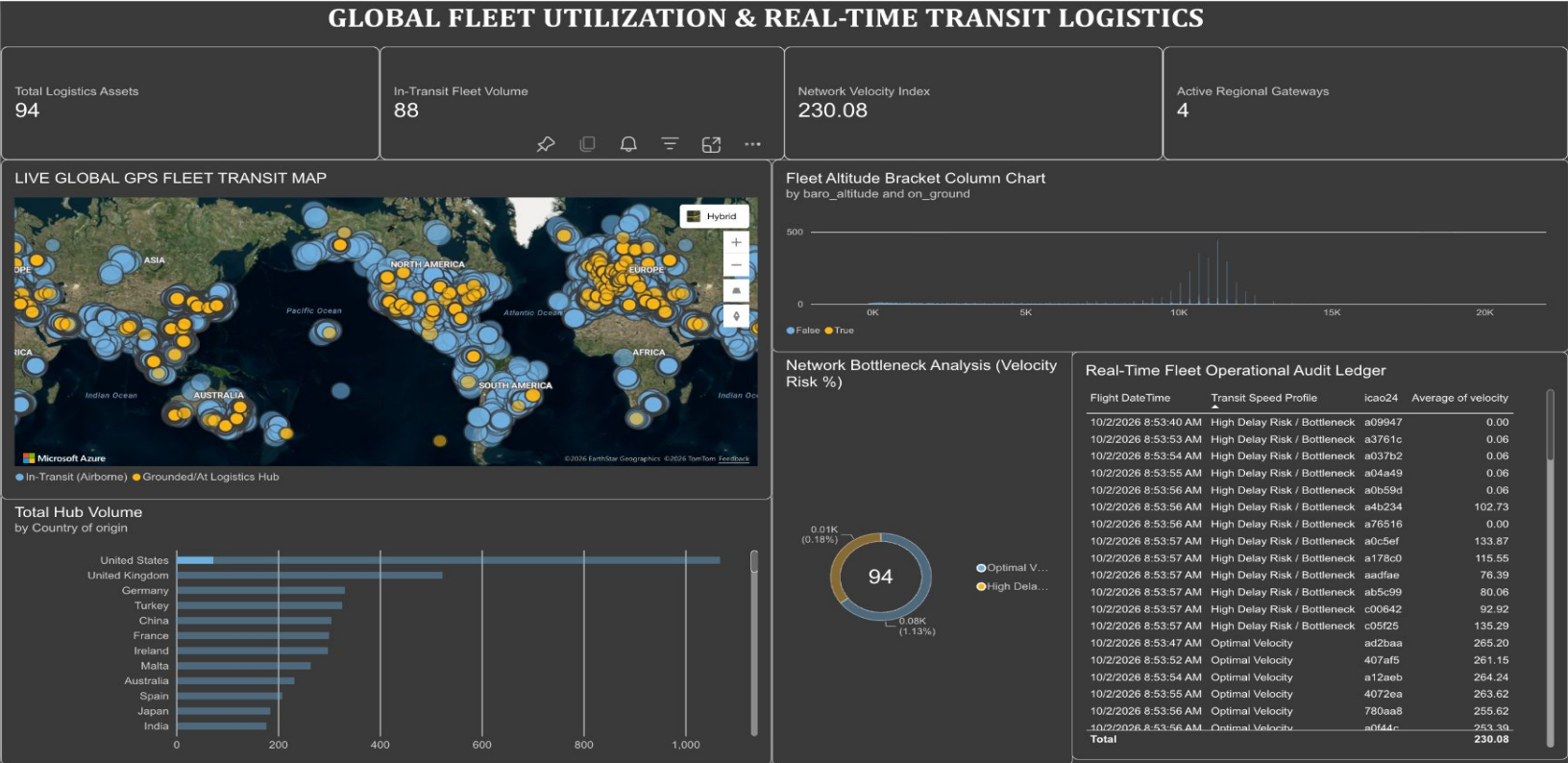


GLOBAL FLEET UTILIZATION & REAL-TIME TRANSIT LOGISTICS DASHBOARD

- **The Business Idea:** Track real-time global flight data to monitor fleet status and catch transit delays early.
- **What I Did:** I built an interactive 8-chart dashboard that displays active asset metrics, live GPS coordinates, and bottleneck risk categories.



Data Pipeline & Warehouse Setup (Python)

- **What I Did:** I coded a 3-part Python pipeline that pulls live OpenSky API telemetry, cleans missing GPS data, and injects a custom speed classification logic using Pandas.
- **Storage & Automation:** The scripts automatically manage local JSON caching to prevent server timeout losses and instantly upsert the structured records into a SQLite database.

```
import os
import json
import requests
import pandas as pd

url = "https://opensky-network.org/api/states/all"
filename = "live_transit_fleet_batch.csv"
cache_file = "opensky_raw_cache.json"

try:
    response = requests.get(url, timeout=10)

    if response.status_code != 200:
        print(f"Server Busy - (Code {response.status_code}).")
        if os.path.exists(cache_file):
            with open(cache_file, 'r') as f:
                raw_flights = json.load(f)
        else:
            raise Exception("No backup cache.")
    else:
        data = response.json()
        raw_flights = data.get('states', [])
        with open(cache_file, 'w') as f:
            json.dump(raw_flights, f)

    columns = [
        'icao24', 'callsign', 'origin_country', 'time_position',
        'last_contact', 'longitude', 'latitude', 'baro_altitude',
        'on_ground', 'velocity', 'true_track', 'vertical_rate',
        'sensors', 'geo_altitude', 'squawk', 'spi', 'position_source'
    ]

    df = pd.DataFrame(raw_flights, columns=columns)

    # Drop rows that don't have location coordinates
    df = df.dropna(subset=['latitude', 'longitude'])

    # Clean up text codes and format as strings
    df['icao24'] = df['icao24'].astype(str)
    df['callsign'] = df['callsign'].astype(str).strip()
    df['origin_country'] = df['origin_country'].astype(str)
    df.insert(0, 'index_id', range(1, len(df) + 1))

    df.to_csv(filename, index=False)

except Exception as e:
    print(f"Error: {e}")
```

```
import requests
import pandas as pd
import json

def fetch_opensky_batch():

    url = "https://opensky-network.org/api/states/all"
    response = requests.get(url)

    if response.status_code == 200:
        data = response.json()
        states = data.get('states', [])
        columns = [
            'icao24', 'callsign', 'origin_country', 'time_position',
            'last_contact', 'longitude', 'latitude', 'baro_altitude',
            'on_ground', 'velocity', 'true_track', 'vertical_rate',
            'sensors', 'geo_altitude', 'squawk', 'spi', 'position_source'
        ]

        df = pd.DataFrame(states, columns=columns)
        df['callsign'] = df['callsign'].str.strip()
        df['airborne'] = df[(df['on_ground'] == False) & (df['longitude'].notna()) & (df['latitude'].notna())]

        df['airborne']['transit_status'] = df['airborne']['velocity'].apply(
            lambda v: 'Delayed / Bottleneck' if (v and v < 200) else 'On-time / Optimal'
        )

        df['airborne']['asset_efficiency_score'] = (df['airborne']['baro_altitude'] / (df['airborne']['velocity'] + 1)).round(2)

        output_file = "live_transit_fleet_batch.csv"
        df['airborne'].to_csv(output_file, index=False)
        print(f"Len(df_airborne):")
        print(f"Saved file to: {output_file}")

    else:
        print(f"Server status code: {response.status_code}")

if __name__ == "__main__":
    fetch_opensky_batch()
```

```
import sqlite3
import json
import pandas as pd

print("Reading your local JSON flight data...")

# 1. Open your local desktop JSON file instead of making a web call
with open("opensky_raw_cache.json", "r") as f:
    raw_flights = json.load(f)

columns = [
    'icao24', 'callsign', 'origin_country', 'time_position',
    'last_contact', 'longitude', 'latitude', 'baro_altitude',
    'on_ground', 'velocity', 'true_track', 'vertical_rate',
    'sensors', 'geo_altitude', 'squawk', 'spi', 'position_source'
]
df = pd.DataFrame(raw_flights, columns=columns)

# 2. Clean up the rows
df = df.dropna(subset=['latitude', 'longitude'])
df['icao24'] = df['icao24'].astype(str)
df['callsign'] = df['callsign'].astype(str).strip()
df['origin_country'] = df['origin_country'].astype(str)

# 3. Add your 1, 2, 3 row index right at the front
df.insert(0, 'index_id', range(1, len(df) + 1))

# 4. Save directly into your SQL database file
connection = sqlite3.connect("fleet_warehouse.db")
df.to_sql("active_fleet_table", connection, if_exists="replace", index=False)
connection.close()

print("Your local data is now stored inside the 'fleet_warehouse.db' SQL table.")
```

Business Problem

- In aviation logistics, equipment downtime and unexpected transit delays directly lead to broken supply chains and missed delivery deadlines. To fix this, operations teams must constantly track fleet utilization knowing exactly which cargo planes are actively moving in the air versus which ones are sitting idle on the tarmac.
- However, because raw sensor data was trapped in unreadable 10-digit Unix integers (like 1790930469), Power BI could not filter the data chronologically or sort it by hours and minutes. This data limitation created a massive tracking blind spot, making it impossible to separate airborne planes from grounded inventory or flag where shipping streams were backing up.
- Wrote a custom Power Query datetime formula to convert those raw numbers into human-readable calendar dates and precise clock times. Configured and scheduled automated batch updates to stream this live tracking data directly into the dashboard.
- **Key Learnings:** Learned how to seamlessly bridge an automated Python ETL backend with a live, cloud-based Power BI dataset. This project proved that transforming raw telemetry integers into a structured time-series model is critical for resolving high-stakes operational blind spots.